



Universidad Nacional
Federico Villarreal



FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS ESCUELA
PROFESIONAL DE INGENIERÍA DE SISTEMAS

TEMA Y ASIGNATURA:
E2 - AVANCE PROYECTO

SISTEMAS DIGITALES Y ARQUITECTURA DE COMPUTADORAS

ALUMNO:
Murrugarra Retamozo Anderson Jose
Mamani Solorzano Sebastian Ernesto
Lazaro Remaycuna Lujan Carrion
Marceliano Yance Percy Lujan

DOCENTE:
ING. Villafuerte Barreto

CICLO -
SECCIÓN:
V
- B

LIMA
,
PERÚ
2026

1. Introducción

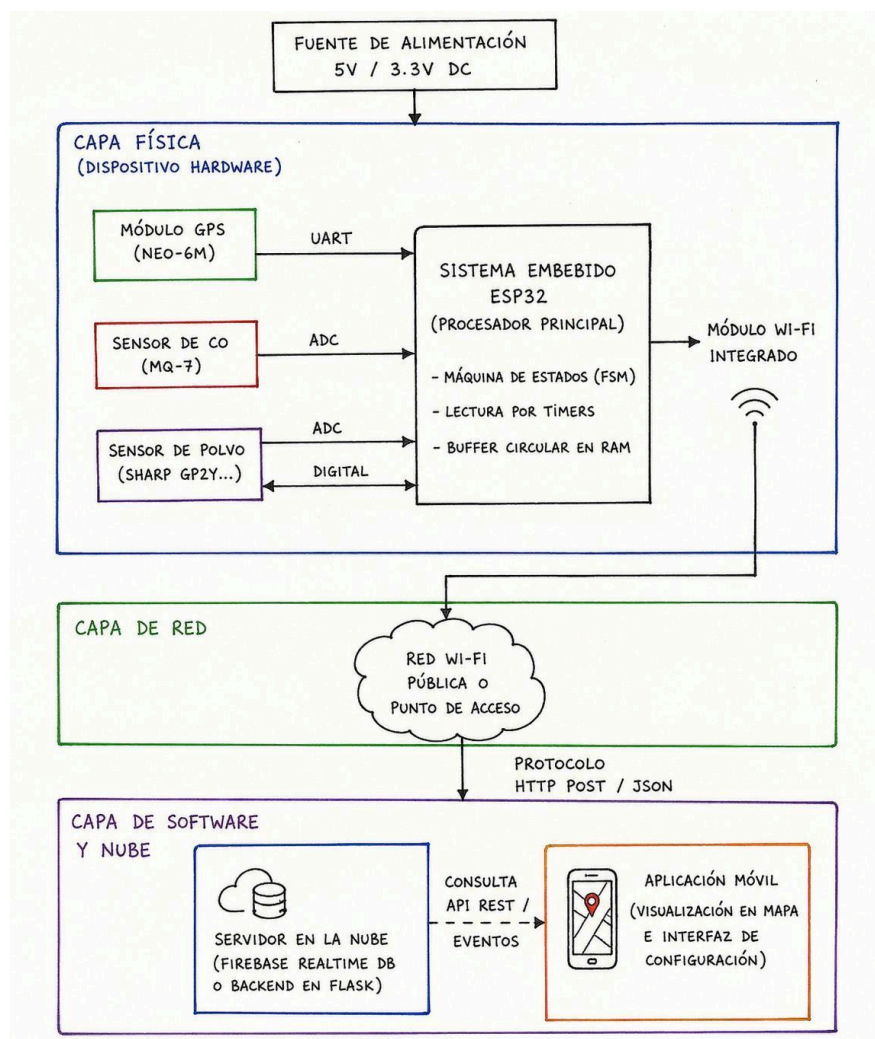
La contaminación del aire representa uno de los principales problemas ambientales y de salud pública a nivel mundial. La exposición prolongada a gases tóxicos, como el monóxido de carbono (CO), puede provocar efectos negativos en la salud de las personas.

Con el objetivo de contribuir al monitoreo ambiental, se desarrolló un sistema basado en Internet de las Cosas (IoT) capaz de detectar concentraciones de monóxido de carbono mediante un sensor MQ-7, obtener la ubicación geográfica utilizando un módulo GPS NEO-6M y transmitir la información a un servidor web mediante conexión WiFi.

El sistema utiliza un microcontrolador ESP32 como unidad principal de procesamiento y comunicación.

WALLY es un dispositivo IoT portátil que mide la concentración de monóxido de carbono (CO) en el aire, registra su ubicación GPS y envía los datos en tiempo real a un servidor en la nube. Está basado en el microcontrolador ESP32, que integra WiFi, procesamiento y adquisición de señales en un solo chip.

DIAGRAMA GENERAL:



2. Objetivos

Objetivo General

Diseñar e implementar un sistema IoT capaz de monitorear la calidad del aire mediante la detección de monóxido de carbono y registrar la ubicación geográfica donde se realiza la medición.

Objetivos Específicos

- Detectar concentraciones de monóxido de carbono utilizando un sensor MQ-7.
- Obtener coordenadas geográficas mediante un módulo GPS NEO-6M.
- Procesar la información mediante un ESP32.
- Transmitir los datos a un servidor remoto mediante WiFi.
- Visualizar la información recopilada para su posterior análisis.

3. Materiales Utilizados

Cantidad	Componente
1	Microcontrolador ESP32 NodeMCU
1	Sensor de gas MQ-7
1	Módulo GPS NEO-6M
1	Protoboard
Varios	Cables Jumper Macho-Macho
Varios	Cables Jumper Hembra-Macho
1	Cable USB a Micro USB
1	Computadora portátil (alimentación y programación)
Futuro	Batería de litio de 5V para alimentación autónoma

4. Descripción del Hardware

ESP32 NodeMCU

Es el microcontrolador principal del sistema. Se encarga de:

- Leer los datos provenientes del sensor MQ-7.

- Comunicarse con el GPS NEO-6M.
- Conectarse a una red WiFi.
- Enviar la información al servidor web.

Sensor MQ-7

El MQ-7 es un sensor diseñado para detectar concentraciones de monóxido de carbono (CO) presentes en el ambiente.

La salida analógica del sensor es leída por el pin GPIO 34 del ESP32.

GPS NEO-6M

Permite obtener:

- Latitud.
- Longitud.
- Número de satélites disponibles.

Se comunica con el ESP32 mediante comunicación serial UART.

Protoboard

Utilizada para realizar las conexiones temporales entre los componentes sin necesidad de soldadura.

5. Conexiones del Circuito

Sensor MQ-7

MQ-7	ESP32
VCC	5V
GND	GND
AO	GPIO 34

GPS NEO-6M

GPS	ESP32
VCC	5V
GND	GND

TX	GPIO 16 (RX)
RX	GPIO 17 (TX)

Alimentación

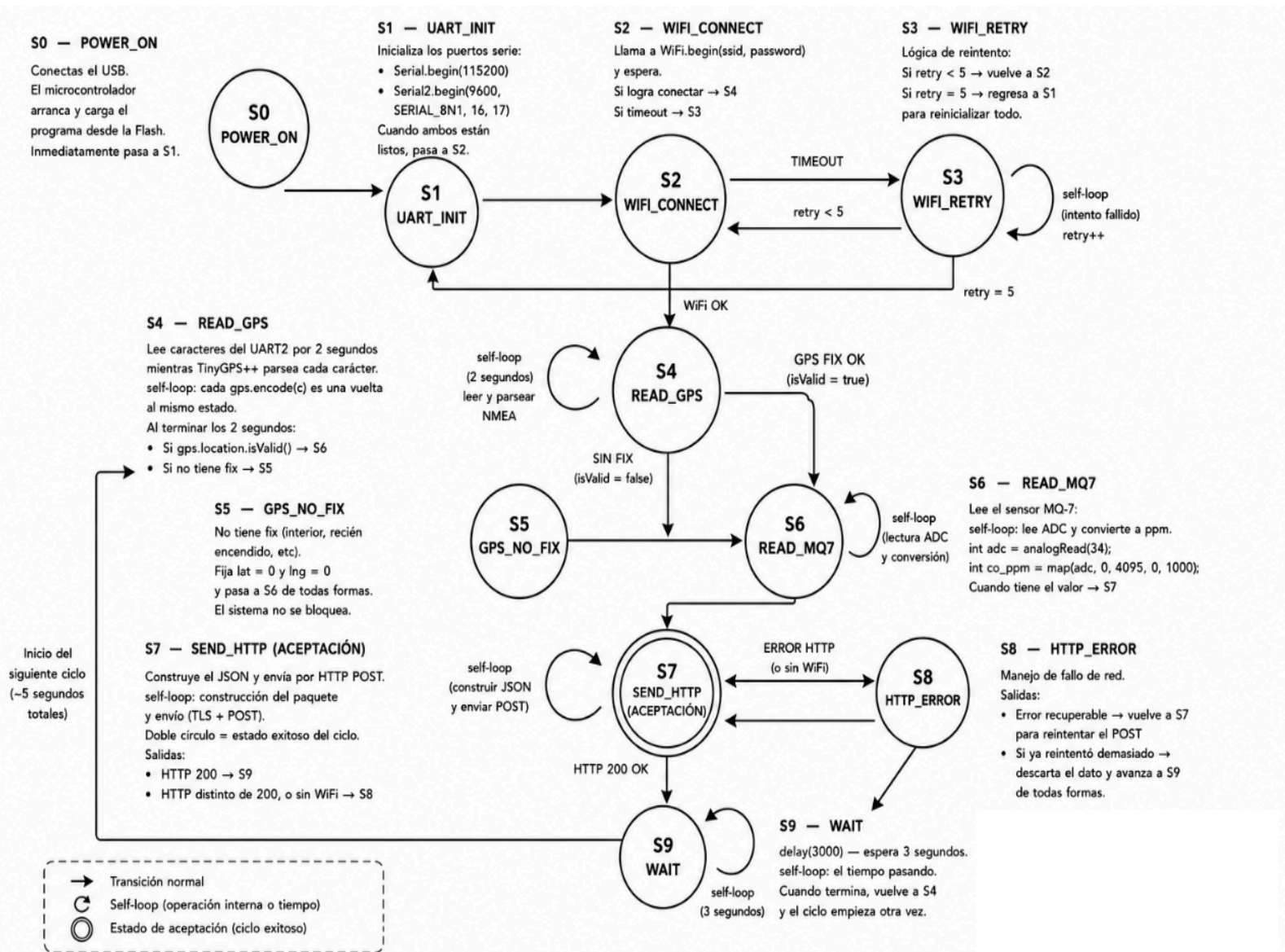
Actualmente:

- El sistema se alimenta mediante el puerto USB conectado a una computadora portátil.

Futuro:

- Se implementará una batería de litio de 5V para permitir la operación autónoma en campo.

6. Máquina de Estados



7. Funcionamiento del Sistema

El funcionamiento general del sistema se desarrolla en las siguientes etapas:

1. Encendido

El ESP32 inicia el programa y configura:

- Comunicación serial.
- Comunicación con el GPS.
- Conexión WiFi.

2. Conexión a Internet

El ESP32 se conecta a la red WiFi configurada.

Una vez establecida la conexión, obtiene una dirección IP para acceder a Internet.

3. Lectura del GPS

Durante aproximadamente dos segundos:

- El GPS envía tramas NMEA.
- El ESP32 procesa la información.
- Se determina la posición geográfica.

Si existen al menos tres satélites visibles y la ubicación es válida, se obtiene:

- Latitud.
- Longitud.

4. Lectura del Sensor MQ-7

El ESP32 realiza una lectura analógica mediante el pin GPIO 34.

El valor digital obtenido se convierte a una escala aproximada de 0 a 1000 ppm de CO utilizando la función map().

5. Construcción del Mensaje JSON

Los datos recopilados son organizados en formato JSON:

```
{  
  "device_id":"WALLY-1",  
  "co":500,  
  "lat":-12.0464,  
  "lng":-77.0428  
}
```

6. Envío al Servidor

Mediante el protocolo HTTP POST se envía la información al

servidor: <https://wally-monitoreo.onrender.com/>

7. Repetición del Ciclo

El proceso se repite cada 3 segundos permitiendo un monitoreo continuo.

8. Arquitectura del Sistema y Mapa de Memoria

Sensor MQ-7

↓

ESP32

↓

WiFi

↓

Servidor Web (Render)

↓

Base de Datos / Plataforma Web

GPS NEO-6M

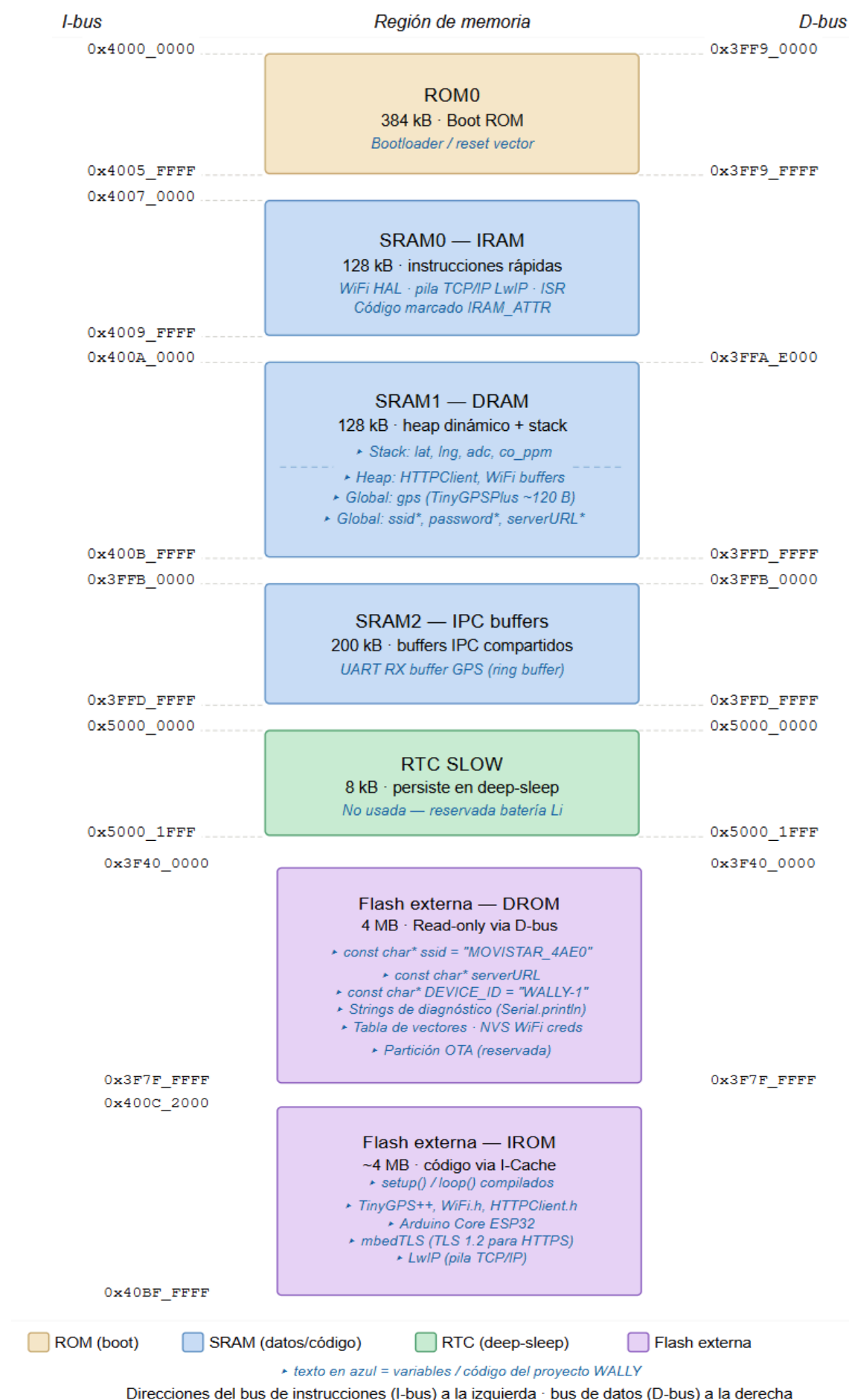
↓

ESP32

↓

Servidor Web

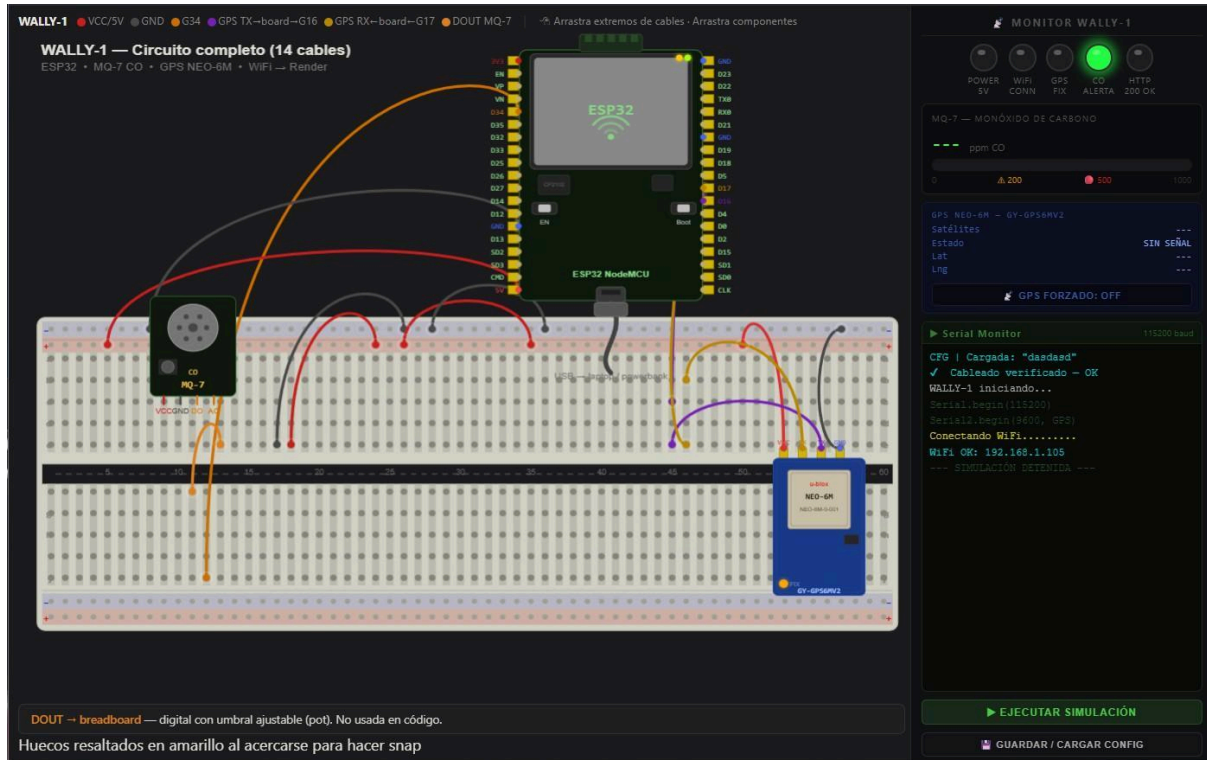
A continuación el Mapa de memoria:



- ROM (naranja) — memoria de solo lectura grabada de fábrica en el chip. Contiene el bootloader que arranca el ESP32. WALLY no escribe nada aquí.
- SRAM0 — IRAM (azul) — RAM para instrucciones de alta velocidad. Aquí vive el WiFi HAL y la pila TCP/IP porque necesitan ejecutarse muy rápido sin pasar por caché.
- SRAM1 — DRAM (azul) — la RAM principal del programa. Aquí están todas tus variables: lat, lng, co_ppm en el stack, y el objeto gps (TinyGPSPlus) y el HTTPClient en el heap dinámico.
- SRAM2 (azul) — buffers compartidos entre los dos núcleos del ESP32. El buffer circular del UART2 (los datos crudos del GPS antes de que TinyGPS++ los procese) vive aquí.
- RTC SLOW / FAST (verde) — memoria que sobrevive al deep-sleep. WALLY no la usa todavía, pero cuando pongas la batería de litio y agregues `esp_deep_sleep_start()`, aquí guardarías el último valor de CO para no perderlo.
- Flash DROM (morado) — la Flash externa de 4 MB accedida por el D-bus. Aquí están todas tus constantes: ssid, password, serverURL, DEVICE_ID, y todos los strings de los `Serial.println()`.
- Flash IROM (morado) — la misma Flash accedida por el I-bus como código. Aquí está todo el código compilado: `setup()`, `loop()`, TinyGPS++, HTTPClient, mbedTLS (el que hace el HTTPS), y LwIP (el TCP/IP).

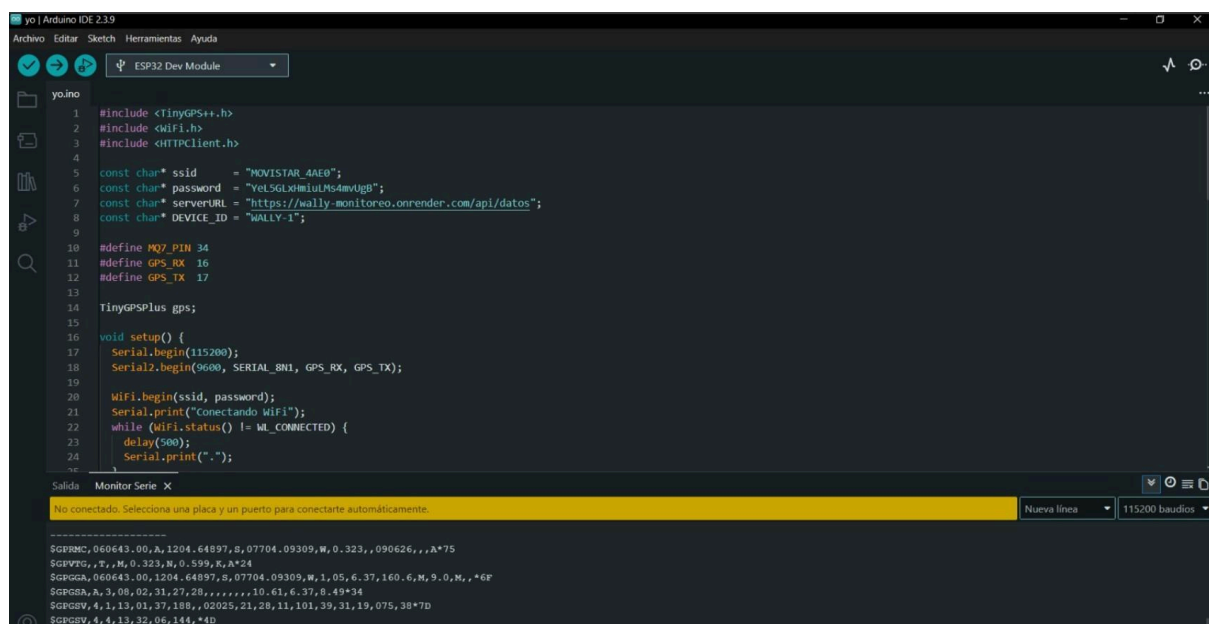
9. Simulación del Circuito

Debido a que no teníamos cómo simular ciertos componentes, gracias a la IA creamos un ejemplo de la simulación del circuito con los componentes y conexiones usadas.



10. IMPLEMENTACIÓN DE LA PÁGINA WEB

Se utilizarán librerías de ESP32 en el Arduino IDE:



CÓDIGO DEL ESP32 COMENTADO:

```
//
```

```
// WALLY-1 — Monitor ambiental móvil  
// Sensores: MQ-7 (CO) + GPS NEO-6M  
// Microcontrolador: ESP32 NodeMCU  
// Envía datos a servidor Render vía HTTP POST (JSON)  
//
```

```
#include <TinyGPS++.h> // Parseo de tramas NMEA del módulo GPS  
#include <WiFi.h>      // Conexión a red WiFi (ESP32 built-in)  
#include <HTTPClient.h> // Cliente HTTP para POST al servidor
```

```
// — Credenciales de red
```

```
— const char* ssid  = "MOVISTAR_4AE0";  
const char* password = "YeL5GLxHmiuLMs4mvUgB";
```

```
// — Endpoint del servidor (Render)
```

```
const char* serverURL =  
"https://wally-monitoreo.onrender.com/api/datos";
```

```
// — Identificador del dispositivo (incluido en cada payload JSON)  
————— const char* DEVICE_ID = "WALLY-1";
```

```
// — Pines
```

```
#define MQ7_PIN 34 // Pin ADC para la salida analógica del MQ-7 (solo lectura)  
#define GPS_RX 16 // RX2 del ESP32 ← TX del GPS NEO-6M  
#define GPS_TX 17 // TX2 del ESP32 → RX del GPS NEO-6M
```

```
// — Objeto TinyGPS++ — acumula y parsea las tramas NMEA —————  
TinyGPSPlus gps;
```

```
//
```

```
// setup() — se ejecuta una sola vez al encender o resetear el ESP32  
//
```

```
void setup() {  
  // Monitor serial USB a 115200 baud para depuración  
  Serial.begin(115200);  
  
  // UART2 a 9600 baud — velocidad estándar del GPS NEO-6M  
  // SERIAL_8N1: 8 bits de datos, sin paridad, 1 bit de parada  
  Serial2.begin(9600, SERIAL_8N1, GPS_RX, GPS_TX);
```

```

// Iniciar conexión WiFi con las credenciales definidas
WiFi.begin(ssid, password);
Serial.print("Conectando WiFi");

// Esperar hasta obtener IP; cada 500 ms imprime un punto de progreso
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

// Confirmar conexión e imprimir la IP asignada por el router
Serial.println("\nWiFi OK: " + WiFi.localIP().toString());
}

//


---


// loop() — se repite indefinidamente; ciclo completo ≈ 2 s GPS + 3 s delay
//


---


void loop() {

    // — 1. LECTURA GPS (ventana de 2 segundos)
    

---


    // Se leen todos los bytes disponibles en Serial2 durante 2 segundos.
    // Cada byte se pasa a gps.encode() para que TinyGPS++ construya
    // internamente la posición, velocidad, satélites, etc.
    // Serial.write(c) vuelca las tramas NMEA crudas al monitor serial.
    unsigned long inicio = millis();
    while (millis() - inicio < 2000) {
        while (Serial2.available()) {
            char c = Serial2.read();
            gps.encode(c);    // alimentar al parser byte a byte
            Serial.write(c);  // eco de tramas NMEA (ej. $GPGGA, $GPRMC...)
        }
    }

    // — 2. LECTURA MQ-7 (CO)
    

---


    // analogRead devuelve 0–4095 (ADC de 12 bits del ESP32).
    // map() convierte ese rango a 0–1000 ppm de CO (escala
    // aproximada). int adc  = analogRead(MQ7_PIN);
    int co_ppm = map(adc, 0, 4095, 0, 1000);

    // — 3. EXTRAER COORDENADAS GPS
    

---


    // Solo se usan si TinyGPS++ tiene un fix válido (mínimo 3 satélites
    // con buena geometría). Si no hay fix, lat y lng quedan en 0.
    float lat = 0, lng = 0;
    if (gps.location.isValid())
        { lat = gps.location.lat();

```

```
    lng = gps.location.lng();  
}
```

// — 4. DIAGNÓSTICO GPS POR SERIAL

```
int sats = gps.satellites.value(); // número de satélites detectados  
Serial.println("\n-----");  
  
if (sats == 0) {  
    // Sin ningún satélite visible — módulo sin señal o en interior  
    Serial.println("GPS | sin señal — mueve hacia la ventana");  
} else if (sats < 3) {  
    // Señal débil: detecta satélites pero insuficientes para calcular posición  
    Serial.print("GPS | señal debil — ");  
    Serial.print(sats);  
    Serial.println(" sat (necesita 3+)");  
} else if (!gps.location.isValid()) {  
    // Suficientes satélites pero el fix aún no está listo (convergiendo)  
    Serial.print("GPS | buscando fix... ");  
    Serial.print(sats);  
    Serial.println(" sat  
detectados");  
} else {  
    // Fix válido — imprime coordenadas con 6 decimales (≈ 11 cm de precisión)  
    Serial.println("GPS | FIX OK");  
    Serial.print("    Lat: "); Serial.println(lat, 6);  
    Serial.print("    Lng: "); Serial.println(lng,  
6);  
}  
  
// Imprimir lectura de CO  
Serial.print("MQ7 | CO: "); Serial.print(co_ppm); Serial.println(" ppm");
```

// — 5. ENVÍO HTTP POST A RENDER

```
— if (WiFi.status() == WL_CONNECTED) {  
    HTTPClient http;  
    http.begin(serverURL); // abrir conexión al endpoint  
    http.addHeader("Content-Type", "application/json"); // indicar formato del  
    cuerpo  
  
    // Construir JSON manualmente con los cuatro campos del payload  
    String body = "{";  
    body += "\"device_id\": \"" + String(DRIVER_ID) + "\", "; // ID del dispositivo  
    body += "\"co\": " + String(co_ppm) + ", "; // ppm de CO (entero)  
    body += "\"lat\": " + String(lat, 6) + ", "; // latitud (6 decimales)  
    body += "\"lng\": " + String(lng, 6); // longitud (6  
    decimales) body += "}";  
  
    // Ejecutar POST y leer código de respuesta HTTP  
    int httpCode = http.POST(body);  
    Serial.print("NET | HTTP: ");  
    Serial.println(httpCode == 200 ? "200 OK" : String(httpCode));  
  
    http.end(); // liberar recursos del cliente HTTP
```

```

} else {
  // Sin WiFi — no se intenta el envío
  Serial.println("NET | sin WiFi");
}

Serial.println("          ");

// Esperar 3 s antes del siguiente ciclo (total ciclo ≈ 5 s con la ventana GPS)
delay(3000);
}

```

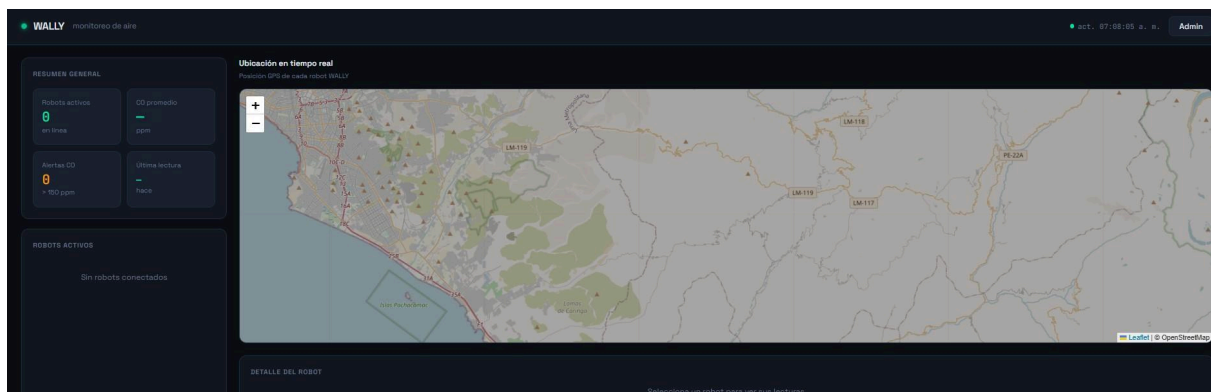
APLICACIÓN WEB: El repositorio del código backend y frontend se encuentr aquí:

https://github.com/perzit0/PROYECTO_WALLY

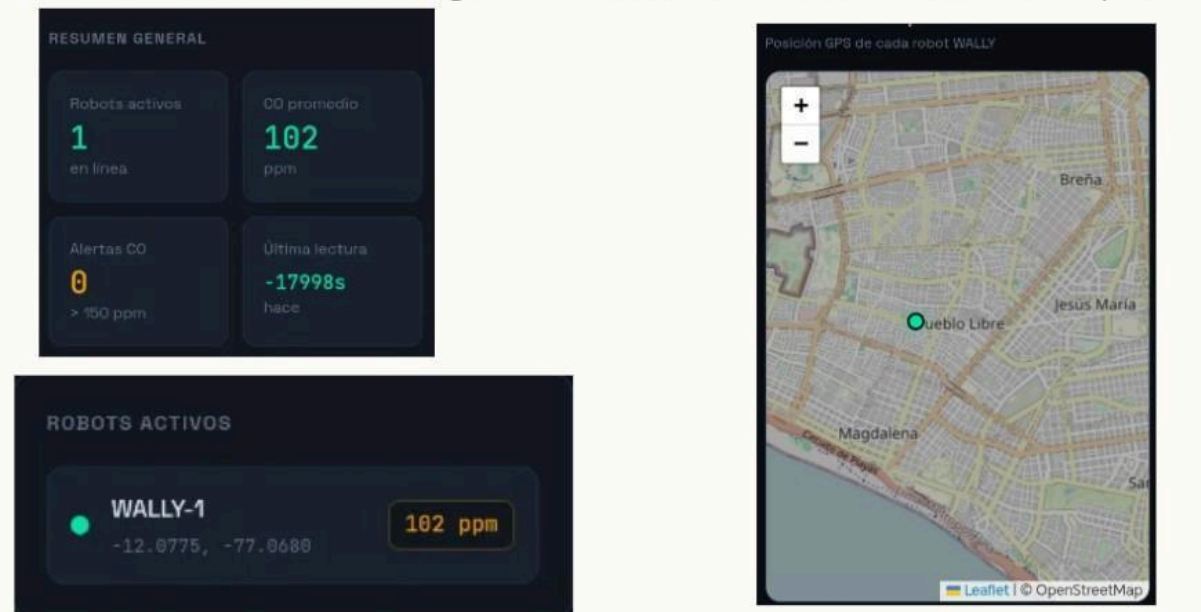
Después de haber creado el backend y el frontend, alojaremos en Render.com, la página web:

<https://wally-monitoreo.onrender.com/>

Después de haber creado el backend y el frontend, alojaremos en Render.com, la página web



Esta nos mostrará el resumen general, los robots activos, la ubicación en tiempo real



10. Resultados Esperados

El sistema permite:

- Detectar niveles de monóxido de carbono.
- Registrar la ubicación exacta donde se realiza cada medición.
- Almacenar la información en un servidor remoto.
- Generar mapas de contaminación ambiental.
- Realizar monitoreo en tiempo real.

11. Ventajas del Proyecto

- Bajo costo de implementación.
- Fácil escalabilidad.
- Monitoreo remoto.
- Registro georreferenciado de datos.
- Posibilidad de alimentar el sistema mediante baterías para uso móvil.
- Integración con plataformas web.

12. Conclusiones

Se desarrolló un sistema IoT capaz de monitorear la calidad del aire mediante la detección de monóxido de carbono utilizando un sensor MQ-7. El sistema integra un módulo GPS NEO-6M para registrar la ubicación geográfica de cada medición y emplea un microcontrolador ESP32 para procesar y transmitir la información a un servidor web.

La solución propuesta permite realizar monitoreo ambiental en tiempo real, facilitando la recopilación de datos georreferenciados que pueden utilizarse para identificar zonas con mayores niveles de contaminación y apoyar futuras estrategias de control ambiental.

13. Anexos

